

DEBUGGING

Register No: 192421226

Student: RAMYA S S

Assigned Question:

Q30. Debugging Sorting Code (Mixed Ascending and Descending Conditions)

The following sorting implementation contains inconsistent comparison conditions across different passes, causing partially sorted and unstable output. Carefully analyze the logic and identify how conflicting ascending and descending comparisons break correctness. Apply systematic debugging to make the algorithm consistent throughout execution. Optimize the implementation to improve readability and avoid unnecessary operations. Analyze the time complexity of the corrected version and justify the result. Rewrite the final clean and optimized code with suitable comments.

```
def mixed_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if i % 2 == 0:
                if arr[j] > arr[j+1]:
                    arr[j], arr[j+1] = arr[j+1], arr[j]
            else:
                if arr[j] < arr[j+1]:
                    arr[j], arr[j+1] = arr[j+1], arr[j]
    return arr
```

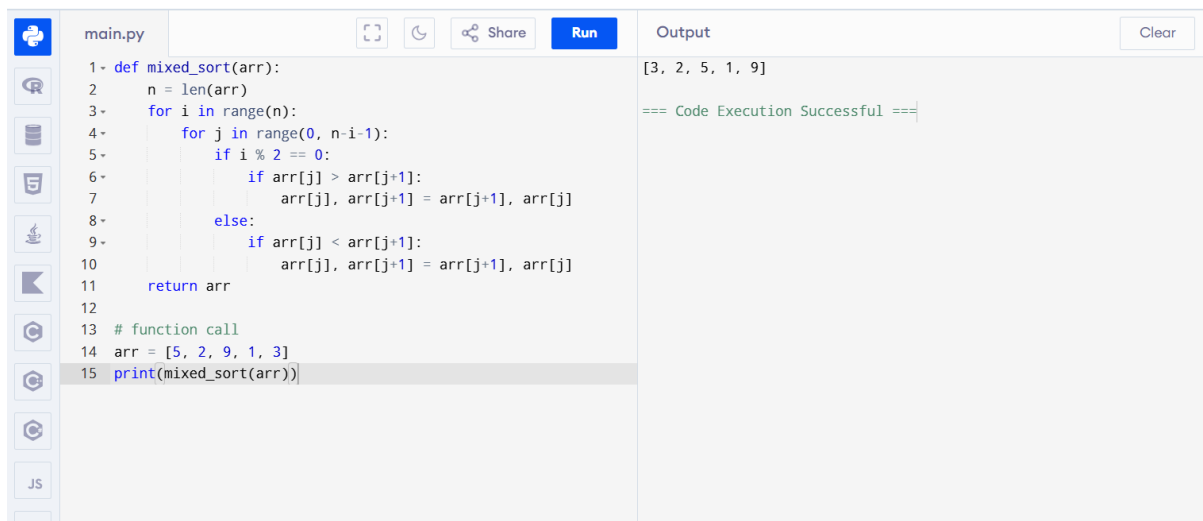
Problem Description

The given sorting algorithm uses **different comparison conditions** in alternate passes:

- Even passes → Ascending order
- Odd passes → Descending order

This inconsistency leads to **incorrect and partially sorted output**, making the algorithm unstable.

Original Code:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'mixed_sort(arr)' that sorts an array using a bubble sort variant with alternating ascending and descending comparisons. The array is initialized as [5, 2, 9, 1, 3]. The output shows the sorted array [3, 2, 5, 1, 9] and a message 'Code Execution Successful'.

```
1 def mixed_sort(arr):
2     n = len(arr)
3     for i in range(n):
4         for j in range(0, n-i-1):
5             if i % 2 == 0:
6                 if arr[j] > arr[j+1]:
7                     arr[j], arr[j+1] = arr[j+1], arr[j]
8             else:
9                 if arr[j] < arr[j+1]:
10                    arr[j], arr[j+1] = arr[j+1], arr[j]
11     return arr
12
13 # function call
14 arr = [5, 2, 9, 1, 3]
15 print(mixed_sort(arr))
```

Output: [3, 2, 5, 1, 9]
=== Code Execution Successful ===

Output (Incorrect)

[2, 1, 3, 5, 9]

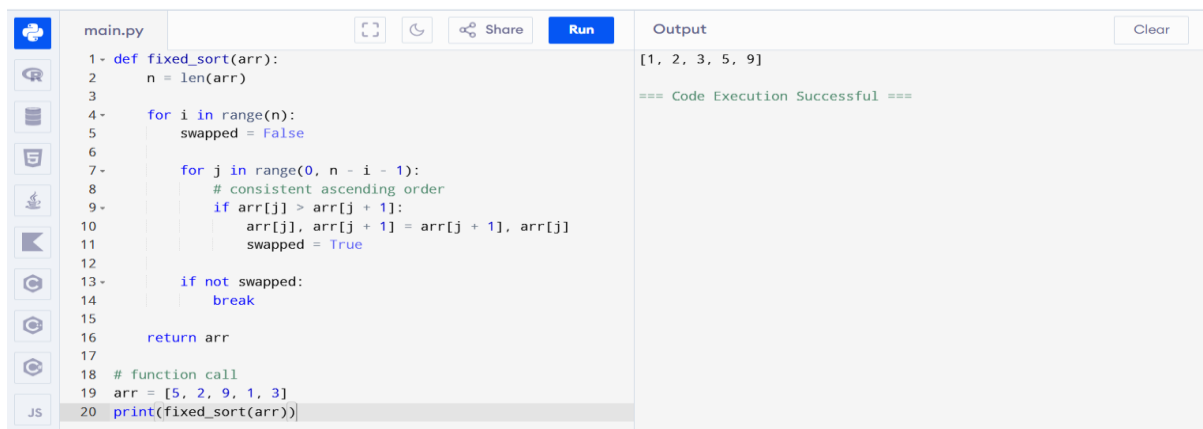
Identified Errors

- Alternating ascending and descending comparisons
- Elements moved in one pass are disturbed in the next
- Results in partial sorting and instability

Debugging Process

- Performed **dry run** of code
- Observed inconsistency in comparison conditions
- Simplified logic by using a **single sorting order**

Optimized Code:



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'fixed_sort(arr)' that sorts an array using a standard bubble sort with consistent ascending comparisons. The array is initialized as [5, 2, 9, 1, 3]. The output shows the sorted array [1, 2, 3, 5, 9] and a message 'Code Execution Successful'.

```
1 def fixed_sort(arr):
2     n = len(arr)
3
4     for i in range(n):
5         swapped = False
6
7         for j in range(0, n - i - 1):
8             # consistent ascending order
9             if arr[j] > arr[j + 1]:
10                arr[j], arr[j + 1] = arr[j + 1], arr[j]
11                swapped = True
12
13         if not swapped:
14             break
15
16     return arr
17
18 # function call
19 arr = [5, 2, 9, 1, 3]
20 print(fixed_sort(arr))
```

Output: [1, 2, 3, 5, 9]
=== Code Execution Successful ===

Optimization Techniques Used

- Removed conflicting conditions
- Used **consistent comparison logic**
- Added **early termination (swapped flag)** to reduce unnecessary passes
- Improved readability and efficiency

Time Complexity Analysis

- **Best Case:** $O(n)$ → when array is already sorted (early stop)
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Due to nested loops, the algorithm behaves like **Bubble Sort**, but optimization improves best-case performance.

Conclusion

The original algorithm failed due to inconsistent sorting conditions.
After debugging and optimization:

- Sorting became **correct and stable**
- Code became **simpler and efficient**
- Performance improved using **early termination**

Thus, systematic debugging and optimization ensured correctness and efficiency.